

Instituto Tecnológico y de
Estudios Superiores de Monterrey.

3. Pokémon TCG Database Modeling Exercise
Software Construction and Decision Making
(Gpo 501)

José Abel Domínguez Rish | A01781852



Tec Campus Santa Fe, 29 de Abril, 2026

Introduction:

This document will describe and justify each entity used in the UML (as proposed in the entity-relationship diagram) for the Pokémon TGC Live game. Each table will include a description of what it represents and its purpose, as well as a list of attributes, including domain definitions. Finally, at the end of the document, there will also be a table listing all the relationships in the diagram, including their constraints.

The generated model has four main areas in the game: the player account and progression system, card placement and deck-building, the economy (including cards, coins, the shop, etc.), and game and match logging.

PLAYER:

Represents each registered user of the game. It is the central entity from which most relationships originate, since a player owns decks, a card collection, a battle pass, currencies and participates in matches. Without this entity it would be impossible to associate game data to a specific user.

Attribute	Type / Domain	Description
id_player	INT (PK)	Unique auto-incremented identifier for each player
username	VARCHAR(50)	Unique display name chosen by the player. Cannot be null.
email	VARCHAR(100)	Unique email address used for authentication. Cannot be null.
level	INT >= 1	Overall player level, calculated from accumulated XP across seasons.

DECK:

Stores the card decks that each player builds. In the game, a player can construct multiple decks with different strategies and formats (Standard or Expanded). This entity allows tracking the composition and format validity of each deck separately from the player's collection.

Attribute	Type / Domain	Description
id_deck	INT (PK)	Unique identifier for the deck.
player_id	INT (FK)	Reference to the player who owns the deck.
name	VARCHAR(100)	Name assigned to the deck by the player (e.g. 'Baraja Charizard').
format	VARCHAR(20)	Format of the deck: 'STANDARD' or 'EXPANDED'. Determines card legality.
created_at	DATE	Date when the deck was created.

COLLECTION:

Each player has exactly one card collection, which contains all the cards they own. This entity serves as an intermediary to implement the 1:1 relationship between PLAYER and their set of cards, allowing a cleaner separation from the DECK entity (which represents constructed decks for play).

Attribute	Type / Domain	Description
id_collection	INT (PK)	Unique identifier for the collection.
player_id	INT (FK)	Reference to the owning player. Unique (each player has exactly one collection).

CARD:

In this table and within the “cards” section, this refers to any base entity, such as a card. It acts as a primary entity within a hierarchy that includes supertypes and subtypes. These can be found in: POKEMON_CARD, TRAINER_CARD, and ENERGY_CARD. After storing the shared attributes, this will prevent duplication, allowing a reference from: DECK_CARD, COLLECTION_CARD, PRIZE_CARD, and GAME_ACTION.

Attribute	Type / Domain	Description
id_card	INT (PK)	Unique identifier for the card across the entire game catalog.
name	VARCHAR(100)	Official card name (e.g. 'Charizard ex', 'Boss's Orders').
card_type	VARCHAR(20)	Card category: 'POKEMON', 'TRAINER', or 'ENERGY'.
rarity	VARCHAR(30)	Rarity tier: Common, Uncommon, Rare, Double Rare, Illustration Rare, etc.
set_name	VARCHAR(100)	Name of the expansion set the card belongs to (e.g. 'Obsidian Flames').

POKEMON_CARD:

This extends the CARD table by adding attributes specific to Pokémon cards. These are: HP, type, phase, evolution chain, weakness, resistance, and retreat cost, which are unique to Pokémon and not shared by other cards, such as Energy or Trainer cards. Therefore, this subtype table uses the same PK as the CARD table, meaning it inherits the table by type.

Attribute	Type / Domain	Description
id_card	INT (PK/FK)	Same PK as CARD. References CARD(id_card).
hp	INT > 0	Hit points of the Pokemon. Must be a positive multiple of 10.
pokemon_type	VARCHAR(20)	Type of the Pokemon: Fire, Water, Grass, Lightning, Psychic, etc.
stage	VARCHAR(20)	Evolution stage: 'Basic', 'Stage 1', or 'Stage 2'.
evolves_from	INT (FK)	Self-referencing FK to the previous stage card. Null for Basic Pokemon.
weakness	VARCHAR(20)	Type this Pokemon is weak against (receives double damage).
resistance	VARCHAR(20)	Type this Pokemon resists (receives reduced damage). Often null.
retreat_cost	INT >= 0	Number of Energy cards required to retreat this Pokemon.

TRAINER_CARD:

Like the Pokémon section, this expands the CARD section, but specifically for Trainers: Items, Support, Stadiums, and Tools. These will have a subtype and a description of their effect. These are stored separately, allowing you to look up Trainer cards specifically

Attribute	Type / Domain	Description
id_card	INT (PK/FK)	Same PK as CARD. References CARD(id_card).
trainer_type	VARCHAR(20)	Sub-category: 'Item', 'Supporter', 'Stadium', or 'Tool'.
effect_description	TEXT	Full text description of the card's effect as printed on the card.

ENERGY_CARD:

Like the others, this also expands the CARD section, but specifically for the Energy cards. Energy cards are essential to the game, as they allow Pokémon to use their attacks. Keeping them as a subtype helps maintain design consistency

Attribute	Type / Domain	Description
id_card	INT (PK/FK)	Same PK as CARD. References CARD(id_card).
energy_type	VARCHAR(20)	Type of energy provided: Fire, Water, Grass, Lightning, Psychic, Fighting, Darkness, Metal, Dragon, Colorless, or Special.

EFFECT:

This table stores the various attacks and abilities possessed by Pokémon cards. A single Pokémon card may have multiple attacks and abilities, typically 1 or 2. This entity allows you to track the effects used during the game, which may include damage, energy cost, and whether a coin flip is required.

Attribute	Type / Domain	Description
id_effect	INT (PK)	Unique identifier for the effect.
card_id	INT (FK)	Reference to the Pokemon card that owns this effect.
name	VARCHAR(100)	Name of the attack or ability (e.g. 'Flamethrower', 'Burning Abyss').
effect_type	VARCHAR(20)	Type: 'ATTACK' or 'ABILITY'.
description	TEXT	Full effect text as printed on the card.
damage	INT >= 0	Base damage of the attack. 0 for non-damaging attacks or abilities.
energy_cost	VARCHAR(50)	Energy types and quantities required (e.g. 'Fire, Colorless, Colorless').
requires_coin_flip	BOOLEAN	True if the attack involves a coin flip mechanic.

COLLECTION_CARD:

This table is essential for linking the collection with CARD. It records how many copies of each card each player has. This is necessary because a player can have multiple cards/copies of the same card, so the collection must track the quantities of similar cards to comply with the game rules (where the maximum is 4 copies per card)

Attribute	Type / Domain	Description
id_collection_card	INT (PK)	Unique identifier for the collection entry.
collection_id	INT (FK)	Reference to the player's collection.
card_id	INT (FK)	Reference to the card.
quantity	INT >= 1	Number of copies of this card owned by the player.

DECK_CARD:

This table will be essential for linking DECK and CARD, as it will track how many and which copies of each card are included in a constructed deck. In the Pokémon card game, exactly 60 cards are required, with a maximum of 4 copies of any non-basic Energy card. Therefore, this table will help enforce these restrictions and allow you to view the deck's composition

Attribute	Type / Domain	Description
id_deck_card	INT (PK)	Unique identifier for the deck entry.
deck_id	INT (FK)	Reference to the deck.
card_id	INT (FK)	Reference to the included card.
quantity	INT 1..4	Number of copies of this card in the deck (1 to 4 for most cards).

MATCH:

Represents a single game session between two players. Stores the outcome information (winner and win condition) and date. The winner is referenced via MATCH_PLAYER to maintain consistency with the match participation model rather than pointing directly to PLAYER.

Attribute	Type / Domain	Description
id_match	INT (PK)	Unique identifier for the match.
match_date	DATETIME	Date and time when the match took place.
winner_match_player_id	INT (FK)	Reference to the MATCH_PLAYER record of the winning player. Null if draw.
win_condition	VARCHAR(50)	How the match was won: 'PRIZES', 'DECK_OUT', 'CONCEDE', or 'TIMEOUT'.

MATCH_PLAYER:

Junction table linking MATCH and PLAYER. Since every match has exactly 2 players, this table stores per-player match data such as the deck used, result, and prize card progress. Referencing winner_match_player_id from MATCH to this table (rather than to PLAYER) avoids ambiguity and keeps all match context together.

Attribute	Type / Domain	Description
id_match_player	INT (PK)	Unique identifier for this player-in-match record.
match_id	INT (FK)	Reference to the match.
player_id	INT (FK)	Reference to the participating player.
deck_id	INT (FK)	Reference to the deck used by this player in this match.
result	VARCHAR(10)	Outcome for this player: 'WIN', 'LOSS', or 'DRAW'.
prize_cards_taken	INT 0..6	Number of prize cards taken by this player during the match.
prize_cards_remaining	INT 0..6	Prize cards still remaining for this player at match end.

TURN:

Records each turn within a match, associated with the player whose turn it was via MATCH_PLAYER. This entity is the temporal backbone of the game log, allowing reconstruction of the full sequence of events. Game actions and prize card claims are linked to specific turns.

Attribute	Type / Domain	Description
id_turn	INT (PK)	Unique identifier for the turn.
match_id	INT (FK)	Reference to the match this turn belongs to.
match_player_id	INT (FK)	Reference to the player whose turn it is.
turn_number	INT >= 1	Sequential turn number within the match (alternates between players).

GAME_ACTION:

Logs every discrete action performed during a turn: playing a card, using an attack, attaching energy, etc. This table enables full replay of a match and analysis of gameplay patterns. It references both the turn and the card involved, with optional effect and damage tracking.

Attribute	Type / Domain	Description
id_action	INT (PK)	Unique identifier for the action.
turn_id	INT (FK)	Reference to the turn in which this action occurred.
player_id	INT (FK)	Reference to the player who performed the action.
card_id	INT (FK)	Reference to the card involved in the action. Nullable for non-card actions.
effect_id	INT (FK)	Reference to the effect used (e.g. a specific attack). Nullable.
action_type	VARCHAR(30)	Type of action: PLAY_CARD, ATTACK, ATTACH_ENERGY, RETREAT, DRAW, etc.
target	VARCHAR(50)	Describes the target of the action (e.g. opponent's Active Pokemon).
coin_flip_required	BOOLEAN	Whether a coin flip was required for this action.
coin_flip_result	VARCHAR(10)	Result of the coin flip: 'HEADS', 'TAILS', or null.
damage_done	INT >= 0	Amount of damage dealt to the target. 0 or null for non-damaging actions.

PRIZE_CARD:

Tracks each of the 6 prize cards assigned to a player at the start of a match. In Pokemon TCG, prize cards are face-down cards taken as a reward for knocking out opponent Pokemon. This table records which card occupies each position, when and whether it was taken, enabling accurate prize tracking and game state reconstruction.

Attribute	Type / Domain	Description
id_prize	INT (PK)	Unique identifier for the prize card slot.
match_player_id	INT (FK)	Reference to the player in the match who owns these prize cards.
card_id	INT (FK)	Reference to the card placed as a prize.
position	INT 1..6	Position slot (1 through 6) in the prize card area.
is_taken	BOOLEAN	Whether this prize card has been claimed.
taken_turn_id	INT (FK)	Reference to the turn when this prize was taken. Null if not yet taken.

CURRENCY:

Catalog table for the types of in-game currency. The game has multiple currencies visible in the UI: Gold coins, Crystals, and Gems (shown in the top bar). Keeping them in a separate table rather than hardcoding makes the model extensible and allows meaningful descriptions per type.

Attribute	Type / Domain	Description
id_currency	INT (PK)	Unique identifier for the currency type.
name	VARCHAR(50)	Name of the currency (e.g. 'Gold', 'Crystal', 'Gem').
description	VARCHAR(200)	Description of how this currency is obtained and used.

PLAYER_CURRENCY:

Junction table storing the balance of each currency type for each player. Since a player can hold multiple currencies simultaneously, a separate table is required to avoid multiple nullable columns in PLAYER. This design allows new currency types to be added without schema changes.

Attribute	Type / Domain	Description
id_player_currency	INT (PK)	Unique identifier for this balance record.
player_id	INT (FK)	Reference to the player.
currency_id	INT (FK)	Reference to the currency type.
amount	INT >= 0	Current balance of this currency for this player.

PURCHASE:

Records transactions made by players in the in-game shop. The shop in Pokemon TCG Live allows purchasing booster packs, cosmetics, and other items. This table provides a purchase history per player, linked to their account and recording cost and date.

Attribute	Type / Domain	Description
id_purchase	INT (PK)	Unique identifier for the purchase.
player_id	INT (FK)	Reference to the purchasing player.
item_name	VARCHAR(100)	Name of the purchased item (e.g. 'Obsidian Flames Booster Pack').
cost	INT > 0	Amount spent in the purchase.
purchase_date	DATETIME	Date and time of the transaction.

BATTLE_PASS:

Stores the Battle Pass progression for each player per season. The Battle Pass (Pase de Combate) is a seasonal reward track visible in the game UI where players earn XP and advance through levels to unlock rewards. Tracking this separately per season allows historical season data to be preserved.

Attribute	Type / Domain	Description
id_battle_pass	INT (PK)	Unique identifier for this battle pass record.
player_id	INT (FK)	Reference to the player.
season	VARCHAR(30)	Season identifier (e.g. 'Season 2024-A').
level	INT >= 1	Current level reached in the battle pass for this season.
xp	INT >= 0	Total XP accumulated within this season's battle pass.

Relationships and Constraints:

This table shows all the relationships in the UML diagram, along with their cardinality and integrity constraints.

Relationship	Cardinality	Description	Integrity Constraints
PLAYER → DECK	1:N	A player can create zero or more decks. Each deck belongs to exactly one player.	player_id in DECK cannot be null. Deleting a player cascades to their decks.
PLAYER → COLLECTION	1:1	Each player owns exactly one card collection. Each collection belongs to exactly one player.	player_id in COLLECTION must be unique and not null.
PLAYER → MATCH_PLAYER	1:N	A player can participate in many matches. Each MATCH_PLAYER record belongs to exactly one player.	player_id in MATCH_PLAYER cannot be null.
PLAYER → GAME_ACTION	1:N	A player can perform many game actions. Each action is performed by exactly one player.	player_id in GAME_ACTION cannot be null.
PLAYER → PLAYER_CURRENCY	1:N	A player can hold balances of multiple currency types.	Combination of (player_id, currency_id) must be unique.
PLAYER → PURCHASE	1:N	A player can make zero or more purchases.	player_id in PURCHASE cannot be null.
PLAYER → BATTLE_PASS	1:N	A player can have a battle pass record per season.	Combination of (player_id, season) must be unique.

Relationship	Cardinality	Description	Integrity Constraints
COLLECTION → COLLECTION_CARD	1:N	A collection contains zero or more card entries.	collection_id in COLLECTION_CARD cannot be null.
CARD → COLLECTION_CARD	1:N	A card can appear in many different collections.	card_id in COLLECTION_CARD cannot be null.
DECK → DECK_CARD	1:N	A deck contains one or more card entries (60 cards total).	deck_id in DECK_CARD cannot be null. Sum of quantities per deck must equal 60.
CARD → DECK_CARD	1:N	A card can be included in many different decks.	Combination of (deck_id, card_id) must be unique.
CARD → POKEMON_CARD	1:0..1	A card is optionally a Pokemon card (supertype/subtype pattern).	id_card in POKEMON_CARD must exist in CARD and card_type must be 'POKEMON'.
CARD → TRAINER_CARD	1:0..1	A card is optionally a Trainer card.	id_card in TRAINER_CARD must exist in CARD and card_type must be 'TRAINER'.
CARD → ENERGY_CARD	1:0..1	A card is optionally an Energy card.	id_card in ENERGY_CARD must exist in CARD and card_type must be 'ENERGY'.
POKEMON_CARD → POKEMON_CARD	0..1:N	A Pokemon card can evolve from another Pokemon card (self-referencing).	evolves_from must reference a card with a lower or equal stage. Cannot form cycles.
CARD → EFFECT	1:0..N	A Pokemon card can have zero or more effects (attacks/abilities).	card_id in EFFECT must reference a card whose id_card exists in POKEMON_CARD.
MATCH → MATCH_PLAYER	1:2	A match always has exactly two player participants.	Exactly 2 MATCH_PLAYER records must exist per match_id. match_id cannot be null.
MATCH_PLAYER → TURN	1:N	Each player takes many turns over the course of a match.	match_player_id in TURN cannot be null.
MATCH_PLAYER → DECK	1:1	Each player uses exactly one deck per match.	deck_id in MATCH_PLAYER cannot be null. The deck must belong to the same player.

Relationship	Cardinality	Description	Integrity Constraints
MATCH_PLAYER → PRIZE_CARD	1:6	Each player is assigned exactly 6 prize cards at match start.	Exactly 6 PRIZE_CARD records per match_player_id. Positions 1-6 must be unique per player.
TURN → GAME_ACTION	1:N	A turn contains zero or more game actions.	turn_id in GAME_ACTION cannot be null.
TURN → PRIZE_CARD (taken)	1:0..N	A turn may result in zero or more prize cards being claimed.	taken_turn_id in PRIZE_CARD is nullable (null = not yet taken).
EFFECT → GAME_ACTION	1:N	An effect can be used in many game actions across different matches.	effect_id in GAME_ACTION is nullable (not all actions use an explicit effect).
CARD → GAME_ACTION	1:N	A card can be referenced in many game actions.	card_id in GAME_ACTION is nullable for turn-ending actions with no specific card.
CARD → PRIZE_CARD	1:N	A card can serve as a prize card in many different matches.	card_id in PRIZE_CARD cannot be null.
CURRENCY → PLAYER_CURRENCY	1:N	A currency type is referenced by many player balance records.	currency_id in PLAYER_CURRENCY cannot be null.

Diagrama:

